Policy Manager Chat Agent Protobuf Contract

For Policy Manager, we can reuse the shared streaming envelope and common types from CM, add `AskPolicyManager`, and define the PM-specific request and structured response.

1. `chat_agent.proto` **— service definition + top-level stream envelope**

- Extend the existing service:

```
service ChatAgent {
  rpc AskCaseManager(AskCaseManagerRequest)
      returns (stream AnswerEvent);

  rpc AskPolicyManager(AskPolicyManagerRequest)
      returns (stream AnswerEvent);
}
```

- `AnswerEvent` — unchanged oneof: `chunk`, `tool_call`, `tool_result`, `payload`, `done` (terminal), `error` (terminal), `ping` (keepalive).
- `AnswerPayload` — extend the existing oneof with `policy_manager_answer_payload`, using a new, unused field number. Preserve the existing Case Manager field.

2. `policy_manager.proto` **— Policy Manager module types**

- `AskPolicyManagerRequest`
  - `request_context` — shared `AgentRequestContext`.
  - `operator_id` — authenticated operator, populated by the backend.
  - `prompt` — user's question.
  - `policy_context` — `PolicyManagerContext`.
  - `history` — repeated `ConversationTurn`.
- `PolicyManagerContext`
  - `policy_set_id` — baseline policy set.
  - `policy_set_version` — exact baseline version.
  - `scenario_id` — historical scenario selected for analysis/backtesting.
  - `case_id` — optional case that prompted the investigation.
  - `candidate_id` — optional existing recommendation being discussed.
  - `candidate_version` — optional version being refined.
  - `simulation_id` — optional run whose results are being discussed.
- `PolicyManagerAnswerPayload`

- `status` — enum: `NEEDS_INPUT`, `CANDIDATE_READY`, `SIMULATION_PENDING`, `EVALUATED`, `INSUFFICIENT_DATA`, `SIMULATION_FAILED`, plus the zero-valued unspecified member.
- `recommendation` — optional `PolicyRecommendation`.
- `simulation` — optional `SimulationSummary`.
- `citations` — repeated `PolicyCitation`.
- `follow_up_questions` — repeated string.
- `limitations` — repeated string.

- `PolicyRecommendation`
  - `candidate_id`, `candidate_version`.
  - `change_type` — enum: `NEW`, `REFINE`, `THRESHOLD_UPDATE`, `MERGE`, `RETIRE`, `NO_CHANGE`, plus unspecified.
  - `target_policy_ids` — repeated affected policy IDs.
  - `title`, `description`, `rationale`.
  - `policy_definition` — optional `PolicyDefinition`.
  - `citation_ids` — repeated supporting citation IDs.
- `PolicyDefinition`
  - `format` — agreed policy representation.
  - `schema_version`.
  - `definition` — serialized policy definition in that format.

  The executable format must match what Veersa's simulator accepts. For `NO_CHANGE` or a request for more information, no executable definition is required.
- `SimulationSummary`
  - `simulation_id`.
  - `status` — enum: `QUEUED`, `RUNNING`, `COMPLETED`, `FAILED`, `CANCELLED`, plus unspecified.
  - `candidate_id`, `candidate_version`.
  - `baseline_policy_set_id`, `baseline_policy_set_version`.
  - `snapshot_id` — historical data evaluated.
  - `baseline_metrics`, `candidate_metrics` — optional `PolicyMetrics`, populated only for completed results.
  - `limitations` — repeated string.
- `PolicyMetrics`

    - `true_positives`, `false_positives`, `true_negatives`, `false_negatives`.
    - `precision`, `recall`, `false_positive_rate` — optional numeric fields; absent when undefined.
- `PolicyCitation`
    - `citation_id`.
    - `source_tool`.
    - `source_record_id` — referenced policy, event, aggregate or simulation record.
    - `snapshot_id` — where applicable.
    - `field_paths` — repeated referenced fields.

3. `common.proto` **— reuse module-neutral shared types**

Reuse:

- `AgentRequestContext` — `tenant`, `organization`, `agent_session_id`, `request_id`.
- `ConversationTurn` — oneof `UserTurn` / `AgentTurn`.
- `Chunk`, `ToolCall`, `ToolResult`, `Done`, `Error`, `Ping`.

Check whether `AgentTurn` contains Case Manager-specific fields; if so, generalize that field to use the shared `AnswerPayload`. Otherwise, no PM-specific changes are needed.

Contract behavior

- The natural-language answer streams through `chunk`; the structured recommendation arrives through `payload`.
- A successful turn emits one `payload`, followed by `done`. `error` terminates the stream without `done`.
- `done` means the **chat turn** finished. A simulation can still be pending.
- Simulation metrics come from Veersa's simulator and must match the referenced candidate version and snapshot.
- This defines the **chat interface**. The agent-to-data and agent-to-simulator RPCs remain separate contracts.

This is a proposed PM schema outline matching your CM structure. Actual field numbers, imports and enum compatibility need the existing `.proto` files before producing a compile-ready extension.